

Universidad Rafael Landívar
Facultad de Ingeniería
Licenciatura en Ingeniería en Informática y Sistemas
Lenguajes Formales y Autómatas
Docente: Ing. Cesar Rojas

Proyecto No.2 Conversión de AFND a AFD y simulación de autómatas

Estudiantes:
Diego Alessandro López Higueros
Sebastián Alexander Villeda Reyna
Carnés: 1187525 y 1032625

Guatemala 25 de septiembre de 2026

Contenido

- 1 Fundamento teórico y objetivo3
- 2 Acciones del programa3
- 3 Datos de entrada y formato4
- 4 Variables y estructuras de datos5
- 5 Condiciones y reglas de operación7
- 6 Diseño y algoritmos7
- 7 Casos de prueba y resultados16
- 8 Manual de usuario21
- 9 Manual técnico28
- 10 Capturas36

1. Fundamento teórico y objetivo

La segunda fase amplía el programa de AFD para representar Autómatas Finitos No Deterministas, convertirlos mediante construcción de subconjuntos y utilizar el mismo simulador sobre el AFD equivalente. Se conserva la carga por teclado y archivo, la validación, las tablas, el análisis estructural y el historial.

1.1 Representación formal

$$M = (Q, \Sigma, \delta, q_0, F)$$

Componente	Significado
Q	Conjunto finito y no vacío de estados.
Σ	Alfabeto de símbolos de entrada.
δ	Función de transición.
q_0	Estado inicial, perteneciente a Q.
F	Conjunto de estados finales, subconjunto de Q.

En un AFD completo, cada par formado por un estado y un símbolo tiene exactamente un destino. En un AFND, el destino es un conjunto que puede estar vacío o contener uno o varios estados.

$$\delta_{AFD} : Q \times \Sigma \rightarrow Q$$

$$\delta_{AFND} : Q \times \Sigma \rightarrow P(Q)$$

$P(Q)$ representa el conjunto de todos los subconjuntos de Q. El AFND acepta una cadena si existe al menos un recorrido que termina en un estado final después de consumir toda la entrada.

1.2 Alcance de la solución

La aplicación trabaja en consola y utiliza listas, conjuntos y diccionarios. La conversión está implementada en el proyecto, sin paquetes de autómatas. Las transiciones epsilon quedan fuera del alcance; se detectan y se rechazan. La cadena vacía sí puede evaluarse y se acepta cuando el estado inicial es final.

El objetivo es obtener un AFD que reconozca el mismo lenguaje que el AFND original. La aplicación conserva hasta dos autómatas independientes y permite elegir el activo. Cada AFND y su AFD equivalente ocupan un solo espacio. Se admiten dos AFD, dos AFND o uno de cada tipo.

2. Acciones del programa

El menú dispone de quince opciones. Las consultas y evaluaciones utilizan el autómata activo. La carga de un AFND válido inicia su conversión automáticamente.

Núm	Acción	Comportamiento
-----	--------	----------------

1	Crear AFD	Solicita la quintupla por teclado y un destino por transición.
2	Cargar AFD	Lee y valida un archivo .txt.
3	Crear AFND	Admite conjuntos de destinos por teclado.
4	Cargar AFND	Lee el archivo, muestra la tabla y convierte a AFD.
5	Mostrar definición y tabla	Presenta el original y, si existe, su AFD equivalente.
6	Validar estructura	Presenta el reporte y permite completar un AFD con trampa.
7	Convertir AFND	Genera el AFD si falta o consulta la conversión existente.
8	Ver equivalencias	Relaciona S0, S1 y los demás macroestados con sus conjuntos.
9	Ver tabla generada	Muestra las transiciones del AFD obtenido del AFND.
10	Evaluar cadena	Presenta la traza, aceptación o rechazo y motivo.
11	Evaluar lote	Lee una cadena por línea y presenta totales.
12	Consultar historial	Muestra las evaluaciones individuales y por lote.
13	Analizar estructura	Presenta alcanzabilidad, finales alcanzables y vacuidad.
14	Administrar autómatas	Carga, selecciona, consulta o quita uno de dos autómatas.
15	Salir	Finaliza la aplicación.

2.1 Estado de la sesión

Cada carga válida ocupa el primer espacio libre y queda activa, sin borrar el otro autómata. Cada espacio conserva su historial y equivalencias. Si ambos están ocupados, se debe quitar uno antes de cargar otro. Una carga fallida conserva los datos existentes. La opción 7 consulta la conversión sin borrar el historial. La evaluación requiere un AFD válido y completo; los AFND se convierten antes de evaluar.

3. Datos de entrada y formato

Dato	Representación	Validación principal
Opción del menú	str con valor 1 a 15	Se consulta en el diccionario de acciones; no se convierte a entero.

Nombre	str	No vacío en archivo; en modo manual se admite un nombre por defecto.
Tipo	AFD o AFND	Si se declara en archivo, debe coincidir con la opción de carga.
Estados	Identificadores separados por comas	Q no vacío; sin identificadores inválidos.
Alfabeto	Símbolos separados por comas	Un carácter por símbolo; se admiten dígitos.
Inicial	Un identificador	Debe pertenecer a Q.
Finales	Identificadores o conjunto vacío	Todos deben pertenecer a Q.
Transición	origen,símbolo,destinos	Referencias existentes y destinos compatibles con el tipo.
Cadena	str, incluida la cadena vacía	Cada carácter debe pertenecer a Σ .
Ruta	str	Archivo legible con codificación UTF 8.
Completar trampa	Respuesta s o n	Solo s autoriza agregarla.

3.1 Archivo AFND de ejemplo

```

NOMBRE=AFND_EJEMPLO
TIPO=AFND
ESTADOS=q0,q1,q2
ALFABETO=a,b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0,a,q0|q1
q0,b,q0 q1,a,
∅
q1,b,q2 q2,a,q2
q2,b,q2

```

Las cabeceras aceptan : o =. TRANSICIONES: utiliza dos puntos. Las cabeceras preceden al bloque de transiciones. El carácter | separa destinos y $\emptyset$ indica ausencia de destinos. Las líneas vacías y los comentarios completos iniciados con # se omiten al leer autómatas.

4. Variables y estructuras de datos

Variable	Tipo conceptual	Uso
estados y alfabeto	set de str	Evitar duplicados y comprobar pertenencia.

estado_inicial	str	Punto de partida de la evaluación.
estados_finales	set de str	Determinar aceptación.
AFD.transiciones	dict de (str, str) a str	Guardar un destino por par.
AFND.transiciones	dict de (str, str) a set	Guardar cero, uno o varios destinos.
transiciones_conflictivas	dict de pares a set	Registrar múltiples destinos al cargar como AFD.
original	AFD, AFND o None	Conservar el modelo ingresado.
afd_actual	AFD o None	Conservar el modelo que se simula.
nombres	dict de frozenset a str	Identificar subconjuntos sin depender de su orden.
pendientes e indice	list e int	Procesar cada subconjunto una sola vez.
equivalencias	dict de str a frozenset	Relacionar el AFD con el AFND original.
union y finales	set	Acumular destinos y macroestados de aceptación.
estado_actual	str	Guardar el estado durante una simulación.
traza	list de str	Registrar cada movimiento de la cadena.
historial	list de ResultadoEvaluacion	Resultados del autómata de ese espacio.
errores y advertencias	list de str	Informar problemas comprensibles.

4.1 Elección de estructuras

Un conjunto permite unir destinos y eliminar repeticiones de manera directa. Un diccionario permite consultar una transición por su par de claves, sin codificar casos específicos para un autómata. Un frozenset es un conjunto nativo inmutable: {q0, q1} y {q1, q0} generan la misma clave en la conversión.

Las colas de conversión y alcanzabilidad usan una lista con índice. El índice avanza sin eliminar el primer elemento, evitando desplazar el resto de la lista. Los constructores copian las colecciones recibidas para reducir cambios involuntarios por referencias compartidas.

AplicacionAFD.espacios contiene dos posiciones: EspacioAutomata o None. activo guarda el índice 0 o 1, o None si no hay autómatas. Cada espacio contiene original, afd_actual, conversor y simulador.

5. Condiciones y reglas de operación

5.1 Integridad y clasificación

El estado inicial debe pertenecer a Q y los finales deben ser un subconjunto de Q . Todo origen y destino utilizado debe existir, y cada símbolo de transición debe pertenecer al alfabeto. Estas reglas se verifican antes de convertir o simular.

Situación	Clasificación y acción
Estado o símbolo inexistente	Definición inválida. No se instala desde archivo.
Varios destinos en un AFD	Definición no determinista. Se bloquea la simulación y se indica cargar como AFND.
Par sin destino en un AFD	AFD incompleto. La opción 6 permite completarlo con trampa.
Par sin destino en un AFND	Conjunto vacío permitido; no equivale a una transición epsilon.
Un destino para cada par del AFD	Completo si también cumple las reglas de integridad.

5.2 Convenciones del formato

Los estados empiezan con una letra de A a Z, mayúscula o minúscula, o con `_`. Después pueden incluir letras, dígitos y `_`. Esta es una convención del archivo; la teoría no exige ese nombre para los estados. El alfabeto admite caracteres individuales y reserva espacios, coma, `|`, `∅` y marcadores epsilon.

Se permite un alfabeto vacío y un conjunto de finales vacío. En archivo, `FINALES=` expresa que no hay finales. Se eliminan duplicados exactos, pero no se descartan entradas inválidas para simular una definición modificada silenciosamente.

5.3 Trampa y lenguaje vacío

Al completar un AFD se añade un estado no final, se dirigen hacia él los pares faltantes y se agregan bucles para cada símbolo. En la conversión, el subconjunto vacío se incorpora solo cuando se alcanza. En ambos casos, una vez en la trampa no se alcanza la aceptación.

$$\text{Inaccesibles} = Q - \text{Alcanzables}$$

$$\text{Finales alcanzables} = F \cap \text{Alcanzables}$$

El lenguaje es vacío si no existe un final alcanzable desde el estado inicial. Tener estados finales declarados no garantiza que alguna cadena sea aceptada.

6. Diseño y algoritmos

6.1 Relación entre componentes

`AplicacionAFD` coordina la carga y dos objetos `EspacioAutomata`. El cargador obtiene datos, el analizador interpreta el texto y el validador comprueba la integridad. Cada espacio conserva el original, su AFD, el conversor y el simulador con su propio historial.

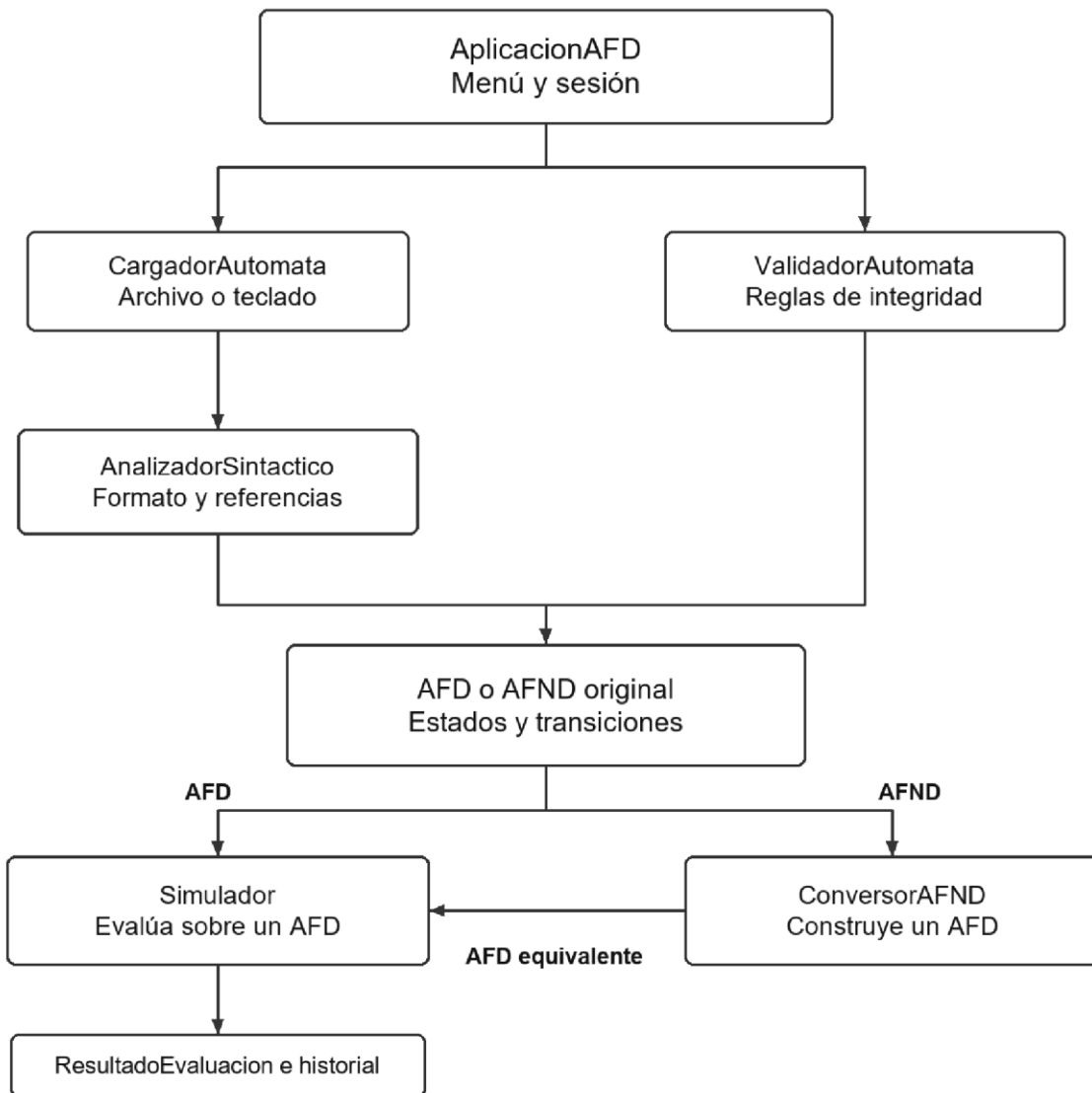


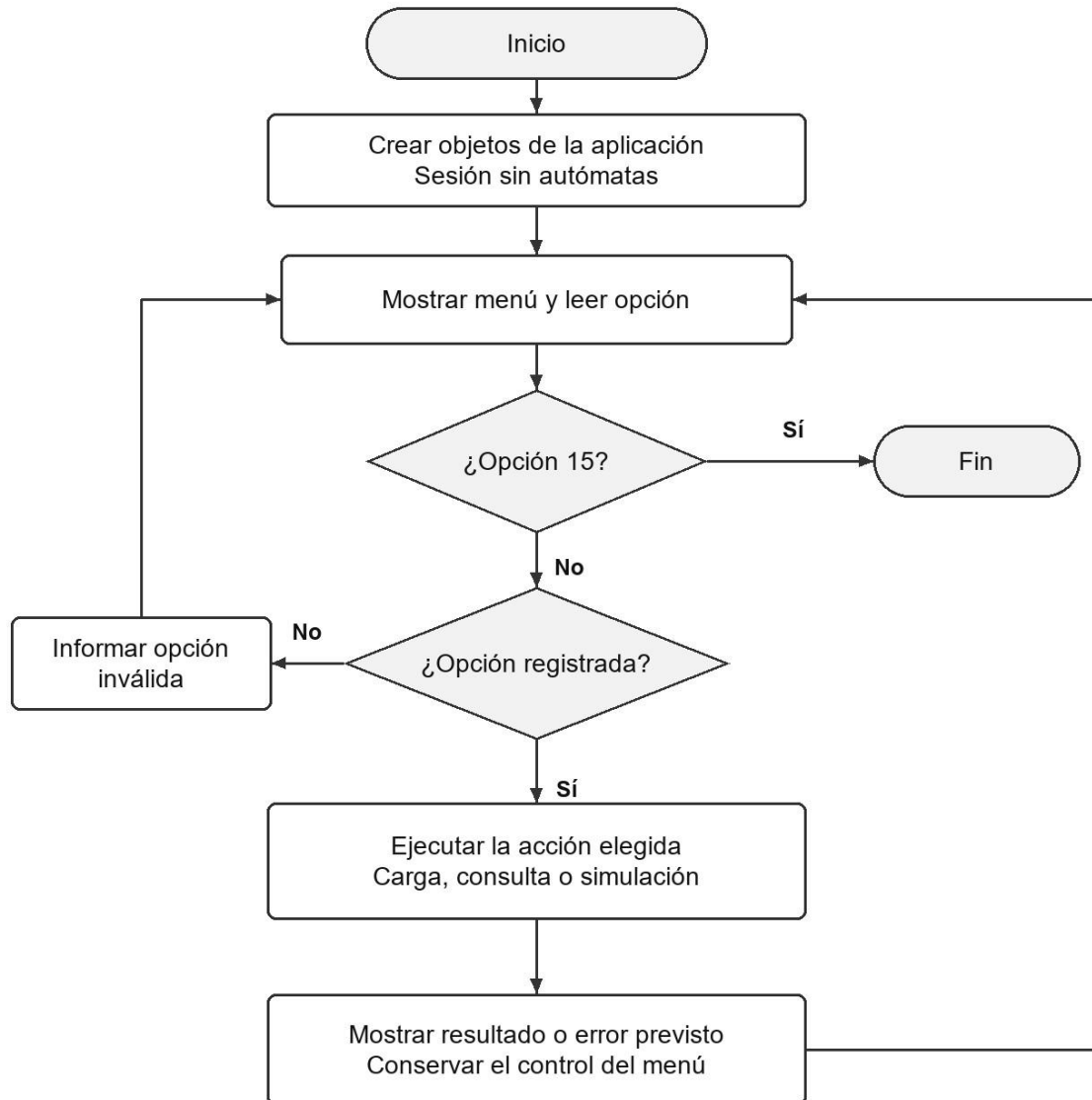
Figura 1 Relación de llamadas y datos entre las clases

Las clases AFD y AFND son independientes; no hay herencia entre ambos modelos. El flujo de conversión y simulación mostrado se aplica al espacio activo. Cada EspacioAutomata conserva su propio conversor, equivalencias y simulador con historial.

6.

2 Flujo general de la aplicación

El diccionario de acciones relaciona cada opción con su método. Los errores previstos se muestran en consola y el control regresa al menú. EOF o una interrupción en el bucle principal finalizan la sesión.



Una carga exitosa reinicia historial y equivalencias.
Una carga fallida conserva la sesión anterior.

Figura 2 Bucle principal y ejecución de las acciones

6.

3 Lectura y validación del archivo

El analizador recorre las líneas por etapas. Un error impide instalar el nuevo autómeta y conserva ambos espacios. Los mensajes mantienen el número de línea cuando corresponde.

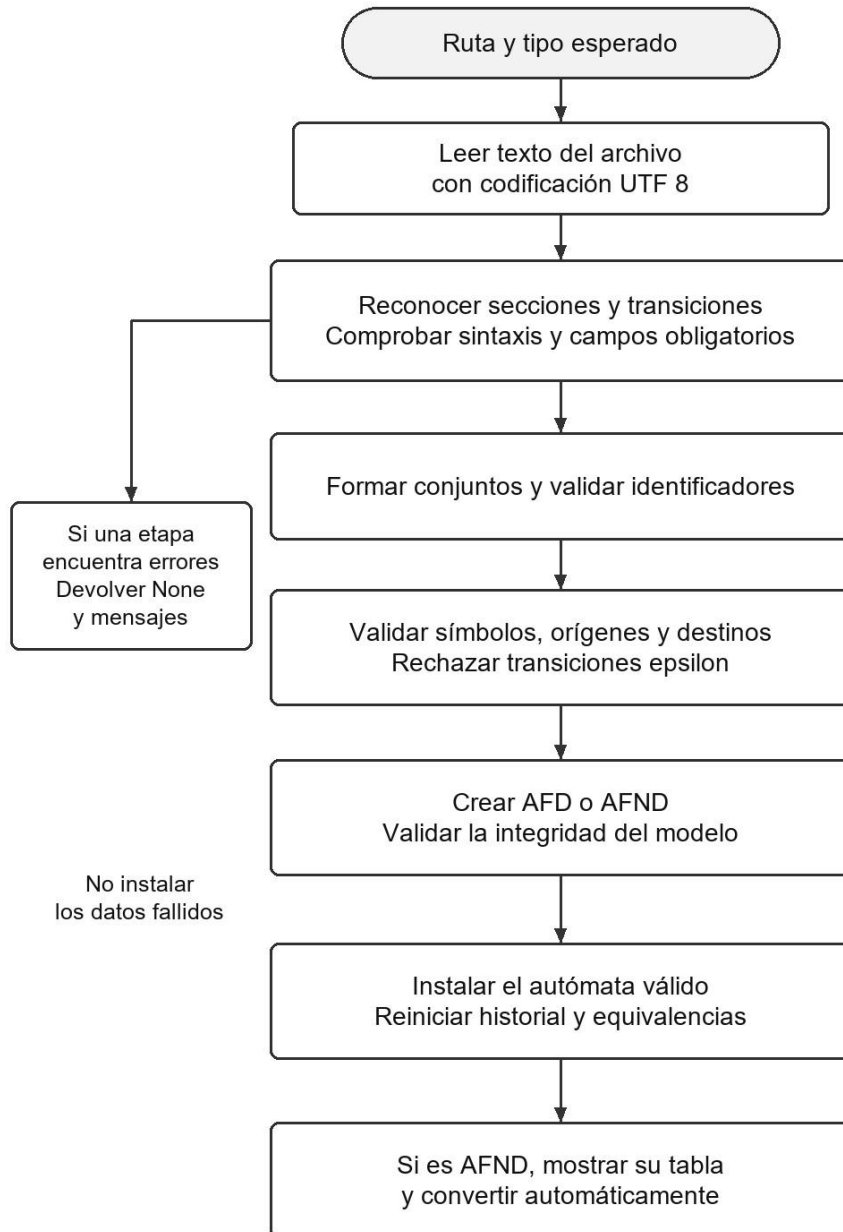


Figura 3 Lectura del archivo y validación antes de instalar el modelo

6.

La revisión se detiene al finalizar una etapa que contiene errores. Por eso un archivo con errores de sintaxis y referencias incorrectas puede requerir corregir la sintaxis antes de ver los problemas restantes.

4 Construcción de subconjuntos

Se parte del conjunto que contiene únicamente al estado inicial del AFND. Para cada conjunto y símbolo se unen todos los destinos. Si el conjunto resultante es nuevo, se registra y se deja pendiente de procesamiento.

6.

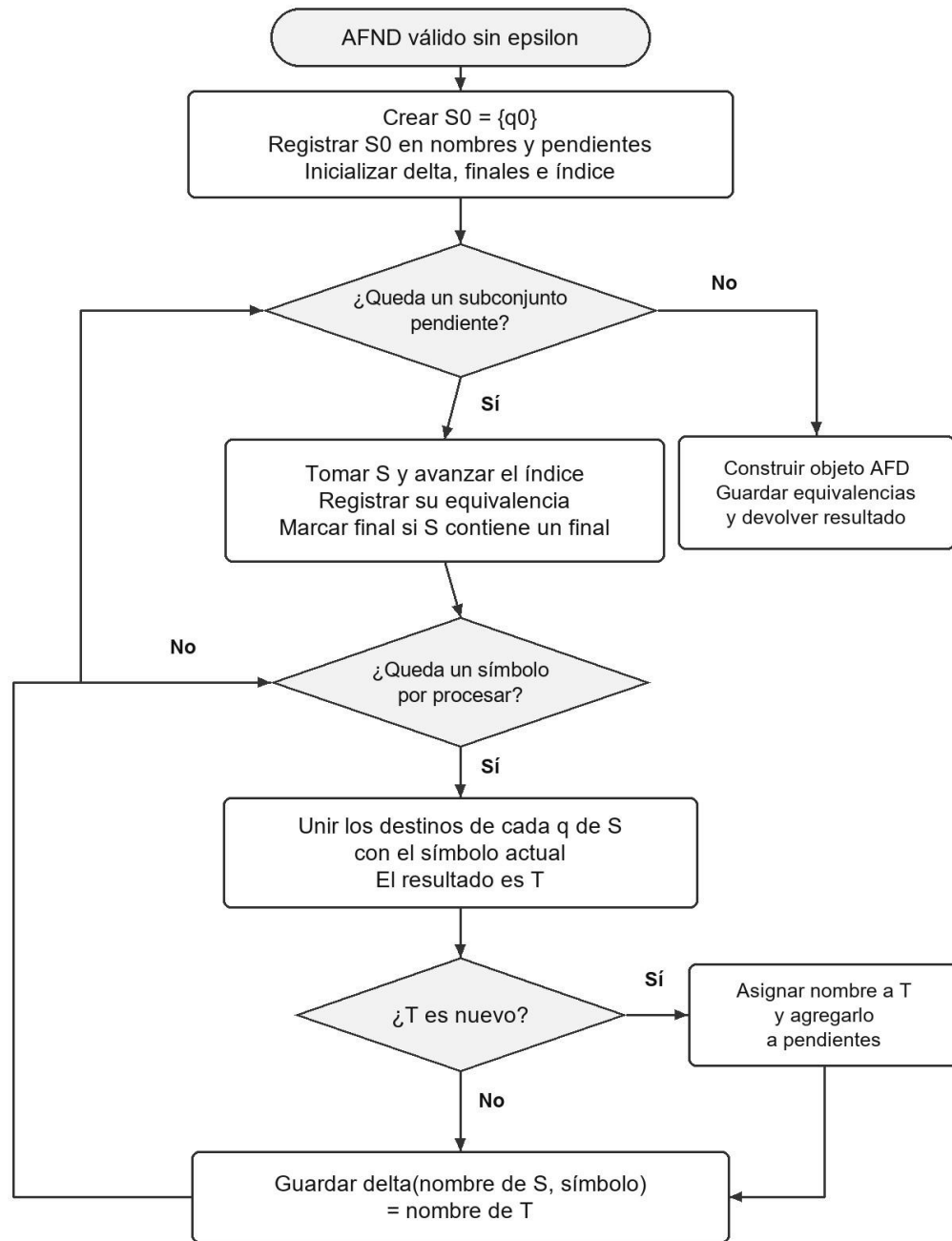


Figura 4 Construcción de los macroestados y del AFD equivalente

5 Desarrollo de la conversión

En afnd_enunciado.txt, el estado inicial es q0 y el único final es q2. El conversor asigna los nombres S0, S1, S2 y S3 en el orden en que descubre los subconjuntos.

6.

Macroestado	Conjunto del AFND	Condición
S0	{q0}	Inicial
S1	{q0, q1}	No final
S2	{q0, q2}	Final porque contiene q2
S3	{q0, q1, q2}	Final porque contiene q2

Desde S0 con a se obtiene {q0, q1}, que se registra como S1. Para procesar S1 con b se consultan dos movimientos: desde q0 se llega a {q0} y desde q1 se llega a {q2}. Su unión es {q0, q2}; ese conjunto recibe el nombre S2.

$$\{q0\} \cup \{q2\} = \{q0, q2\}$$

Estado del AFD	a	b
S0 inicial	S1	S0
S1	S1	S2
S2 final	S3	S2
S3 final	S3	S2

6.5.1 Criterio de aceptación

Un macroestado es final cuando contiene al menos un final del AFND. Esto conserva la regla de aceptación del modelo no determinista: basta con que exista un recorrido de aceptación. No es necesario que todos los estados del conjunto sean finales.

6.5.2 Justificación de equivalencia

Antes de leer símbolos, S0 representa exactamente {q0}. Si un macroestado contiene todos los estados posibles después de un prefijo, la unión de sus destinos contiene todos los estados posibles después del siguiente símbolo. Por inducción sobre la longitud de la cadena, cada recorrido del AFD representa los estados posibles del AFND. Ambos aceptan cuando ese conjunto contiene algún elemento de F.

El ejemplo no genera $\emptyset$ porque ningún movimiento desde un macroestado alcanzable produce un conjunto vacío. El archivo afnd_con_trampa.txt sí produce ese caso.

6.

6 Simulación de una cadena

El simulador recibe un AFD válido y completo. Primero revisa el alfabeto de la cadena y después consulta una transición por carácter. Una cadena vacía no realiza movimientos: acepta si el inicial es final.

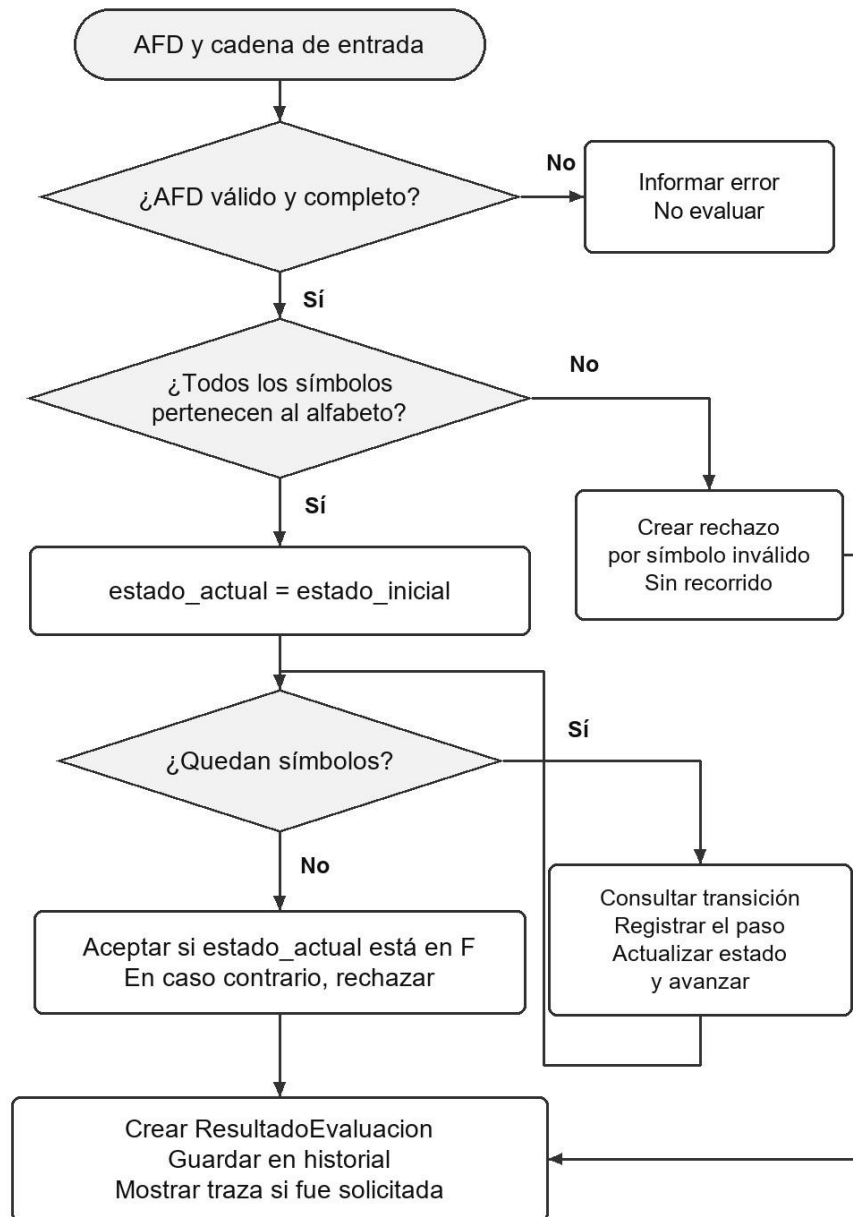


Figura 5 Validación de entrada y simulación sobre el AFD

6.

Un símbolo fuera del alfabeto produce rechazo antes del recorrido. La evaluación individual imprime la traza; el lote conserva los pasos en cada resultado, pero presenta un resumen.

7. Casos de prueba y resultados

Se verificó la versión de código documentada mediante los ejemplos y una ejecución de las comprobaciones automatizadas. Los resultados de esta sección distinguen comportamiento esperado y resultado observado. Las capturas de la sesión del usuario se incorporan en los lugares señalados.

Caso	Entrada o acción	Resultado esperado y observado
CP01	ejemplo_afd.txt	AFD válido y completo. Se mantiene la funcionalidad de la fase 1.
CP02	afd_incompleto.txt y opción 6	Detecta pares faltantes. Agrega q_trampa al responder s.
CP03	afnd_enunciado.txt	Admite q0,a,q0 q1 y conserva los destinos como conjuntos.
CP04	Conversión del ejemplo	Genera cuatro macroestados: S0, S1, S2 y S3.
CP05	afnd_con_trampa.txt	Genera S2 = $\emptyset$ con bucles en a y b; no es final.
CP06	Macroestados S2 y S3 del ejemplo	Son finales y contienen tanto estados finales como no finales.
CP07	ab, b, vacía y abx en el ejemplo	Aceptada, rechazada, rechazada y rechazo por símbolo x.
CP08	afnd_inicial_final.txt y cadena vacía	Aceptada, porque S0 contiene al estado inicial final q0.
CP09	Archivos con errores	Se informa el problema sin instalar la definición inválida.
CP10	Cambio de autómata con opción 14	Conserva ambos modelos e historiales separados.
CP11	Repetir opción 7 tras evaluar	La consulta de conversión conserva el historial.
CP12	AFND y AFD equivalentes	Coinciden los resultados de las cadenas comparadas.

La ejecución automatizada terminó con 12 grupos correctos y 642 comparaciones de aceptación coincidentes. Incluye ejemplos concretos y todas las 16 relaciones de transición sobre dos estados y un símbolo, combinadas con los cuatro conjuntos de finales, para longitudes de 0 a 4. Las pruebas finitas apoyan la implementación; la equivalencia general se justifica con el algoritmo descrito en 6.5.2.

La administración de dos espacios se comprueba con los casos adicionales de la sección 10.2. Las capturas 8, 9 y 10 documentan la carga simultánea, los historiales independientes y la eliminación con reemplazo.

7.1 Lotes y manejo de errores

Autómata	Archivo de cadenas	Total	Aceptadas	Rechazadas
ejemplo_afd.txt	ejemplo_cadenas.txt	5	3	2
Prueba1.txt	prueba2.txt	7	5	2

prueba3.txt	prueba4.txt	8	3	5
prueba7.txt	prueba8.txt	6	2	4
afnd_enunciado.txt	cadenas_fase2.txt	6	3	3

Cada lote se ejecuta con el autómata de su fila. Los archivos de cadenas se leen desde la opción 11; no contienen la definición de un autómata. En cadenas_fase2.txt la tercera línea está vacía y representa una cadena de longitud cero.

Archivo	Resultado observado
prueba5.txt	Rechaza los símbolos 10, 20 y 30 porque tienen más de un carácter.
prueba6.txt	Rechaza los identificadores numéricos según la convención del formato; informa líneas 2 y 5.
afd_prueba_errores.txt	Reporta primero el error de sintaxis de la línea 12. Los demás errores requieren superar esa etapa.
error_sintaxis.txt	Reporta la línea 8, donde la transición usa punto y coma.
error_referencias.txt	Reporta destino q9 en línea 8, símbolo x en línea 9 y origen q9 en línea 10.
error_epsilon.txt	Reporta epsilon en la línea 8 como fuera del alcance.

7.1.1 Casos de control de sesión

Sin un autómata activo, las opciones que dependen de él, incluidas 8 y 12, piden cargar uno. Una opción desconocida, una ruta inexistente o un tipo incompatible producen mensajes y no terminan el menú.

Un archivo incorrecto conserva ambos espacios. Con un AFD cargado directamente, las opciones 7 y 9 requieren un AFND original. Si los dos espacios están ocupados, una nueva carga se bloquea hasta quitar uno.

7.2 Comparación entre AFND y AFD

La siguiente comparación utiliza afnd_enunciado.txt. En la columna del AFND se anotan los conjuntos posibles después de cada símbolo; en la del AFD se anotan los macroestados correspondientes. El estado inicial se incluye en ambos recorridos.

Cadena	Recorrido del AFND	Recorrido del AFD	Resultado común
E	{q0}	S0	Rechazada
A	{q0} → {q0,q1}	S0 → S1	Rechazada
ab	{q0} → {q0,q1} → {q0,q2}	S0 → S1 → S2	Aceptada

B	$\{q_0\} \rightarrow \{q_0\}$	$S_0 \rightarrow S_0$	Rechazada
aab	$\{q_0\} \rightarrow \{q_0, q_1\} \rightarrow \{q_0, q_1\} \rightarrow \{q_0, q_2\}$	$S_0 \rightarrow S_1 \rightarrow S_1 \rightarrow S_2$	Aceptada
bab	$\{q_0\} \rightarrow \{q_0\} \rightarrow \{q_0, q_1\} \rightarrow \{q_0, q_2\}$	$S_0 \rightarrow S_0 \rightarrow S_1 \rightarrow S_2$	Aceptada

abx está fuera de Σ^* porque x no pertenece al alfabeto. El programa la rechaza en la comprobación previa; no constituye una cadena del lenguaje sobre la cual continuar el recorrido.

7.2.1 Comparación con conjunto vacío

En afnd_con_trampa.txt, la cadena b lleva al AFND de $\{q_0\}$ a $\emptyset$ y al AFD de S_0 a S_2 . Ambos rechazan. La cadena ab llega a $\{q_2\}$ y a S_3 , respectivamente; ambos aceptan. Desde $\emptyset$ no se puede recuperar un recorrido de aceptación.

7.2.2 Final inicial

En afnd_inicial_final.txt, la cadena vacía conserva $\{q_0\}$ en el AFND y S_0 en el AFD. Ambos aceptan porque q_0 es final. El caso comprueba que aceptar ϵ no requiere implementar transiciones epsilon.

```
Cargado: AFND_inicial_final
AFND válido (sin epsilon)
ESTADO | a      | b
->*q0  | {q0, q1} | ∅
q1    | {q1}     | ∅
-> inicial; * final; ∅ sin destinos
```

Conversión terminada.

```
MACROESTADO | ESTADOS DEL AFND
S0          | {q0}
S1          | {q0, q1}
S2          | ∅
```

ESTADO	a	b
->*S0	S1	S2
*S1	S1	S2
S2	S2	S2

Leyenda: '->' estado inicial, '*' estado final, '-' transición no definida

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===

1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir

Opción: 10

Cadena (ENTER: cadena vacía): ab

--- TRAZA DE EJECUCIÓN: 'ab' ---

[S0] --(a)--> [S1]

[S1] --(b)--> [S2]

Veredicto final: RECHAZADA

Motivo: El estado final de la computación 'S2' NO pertenece a F.

```
=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 10
Cadena (ENTER: cadena vacía):

--- TRAZA DE EJECUCIÓN: 'ε (cadena vacía)' ---
(Sin pasos: la cadena es vacía o fue rechazada antes de simular)
Veredicto final: ACEPTADA
Motivo: El estado final de la computación 'S0' pertenece a F.
```

8. Manual de usuario

8.1 Requisitos y ejecución

Se requiere Python 3.8 o superior y una terminal. La aplicación no necesita instalar paquetes externos. Los módulos estándar utilizados por el programa son `re`, `os` y `typing`; las demás clases pertenecen al proyecto.

Proyecto_Lenguajes/	Programa/
afd.py	afnd.py
analizador.py	cargador.py
conversor.py	main.py
simulador.py	validador.py
Ejemplos/	archivos de autómatas y
de cadenas	
Pruebas/	comprobaciones opcionales

Abrir la terminal en la carpeta principal del proyecto y ejecutar:

```
python Programa/main.py
```

Las rutas relativas se interpretan desde la carpeta en la que se abrió la terminal. Desde la raíz del proyecto se utiliza `Ejemplos/afnd_enunciado.txt`. Si la terminal está dentro de `Programa`, la ruta es `../Ejemplos/afnd_enunciado.txt`. También se admite una ruta absoluta, incluso entre comillas dobles.

8.2 Primera carga y consulta

Seleccionar 4 e ingresar `Ejemplos/afnd_enunciado.txt`. El programa muestra la tabla del AFND y lo convierte automáticamente. Usar 5 para consultar ambos modelos, 8 para revisar la equivalencia de estados y 9 para ver la tabla del AFD generado. La opción 6 valida la estructura y la 13 presenta su análisis.

Para un AFD de la primera fase, seleccionar 2 y cargar `Ejemplos/ejemplo_afd.txt`. Sus consultas y evaluaciones funcionan con las mismas opciones; no requiere conversión.

```

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
 1. Crear AFD manualmente
 2. Cargar AFD desde archivo
 3. Crear AFND manualmente
 4. Cargar AFND desde archivo
 5. Definición formal y tabla del autómata
 6. Validar estructura / completar AFD con trampa
 7. Convertir AFND a AFD
 8. Tabla de equivalencias de macroestados
 9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 

```

```

4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 4
Ruta del archivo AFND: C:\Users\veget\Downloads\Proyecto_Lenguajes_Fase_2\Proyecto_Lenguajes_Fase_2\Ejemplos\afnd_enunciado.txt

Cargado: AFND_EJEMPLO
AFND válido (sin epsilon)
ESTADO | a      | b
->q0   | {q0, q1} | {q0}
q1     | {}       | {q2}
*q2    | {q2}    | {q2}
-> inicial; * final; {} sin destinos
Conversión terminada.
MACROESTADO | ESTADOS DEL AFND
S0           | {q0}
S1           | {q0, q1}
S2           | {q0, q2}
S3           | {q0, q1, q2}
ESTADO      | a      | b
-----
->S0        | S1     | S0
S1          | S1     | S2
*S2         | S3     | S2
*S3         | S3     | S2

Leyenda: '->' estado inicial, '*' estado final, '-' transición no definida

```

```

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 7
Conversión terminada.
MACROESTADO | ESTADOS DEL AFND
S0           | {q0}
S1           | {q0, q1}
S2           | {q0, q2}
S3           | {q0, q1, q2}
ESTADO      a      b
-----
->S0        S1      S0
S1          S1      S2
*S2         S3      S2
*S3         S3      S2

Legenda: '->' estado inicial, '*' estado final, '-' transición no definida

```

8.3 Creación manual

Las opciones 1 y 3 solicitan nombre, estados, alfabeto, inicial, finales y transiciones. Los conjuntos se escriben separados por comas. Los datos inválidos se pueden corregir en la misma operación. Los duplicados exactos se conservan una sola vez.

Dato	AFD manual con opción 1	AFND manual con opción 3
Nombre	AFD_Manual	AFND_Manual
Estados	q0,q1	q0,q1
Alfabeto	a,b	a,b
Inicial	q0	q0
Finales	q1	q1
delta(q0,a)	q1	q0 q1

delta(q0,b)	q0	ENTER
delta(q1,a)	q1	q1
delta(q1,b)	q0	ENTER

En modo manual, ENTER o $\emptyset$ indica ausencia de destino. En un AFD deja una transición faltante y deberá completarse antes de evaluar. En un AFND representa un conjunto vacío válido. El AFND manual se convierte al terminar su creación.

8.3.1 Resultados de los ejemplos manuales

Cadena	AFD manual	AFND manual convertido
a	Aceptada	Aceptada
aa	Aceptada	Aceptada
ba	Aceptada	Rechazada
ab	Rechazada	Rechazada
Cadena vacía	Rechazada	Rechazada

8.3.2 Corrección y cancelación

Un inicial o final q9 se rechaza si no pertenece a Q. Un símbolo aa se rechaza por su longitud. Un destino q0|q1 se rechaza durante la creación de un AFD, pero es válido en un AFND. Ctrl+C durante la creación manual cancela esa operación y conserva la sesión anterior.

Si no existen estados finales, dejar su respuesta vacía. Un alfabeto vacío también está permitido; en ese caso solo tiene sentido evaluar la cadena vacía, cuya aceptación depende del estado inicial.

8.4 Evaluación e historial

La opción 10 recibe una cadena y muestra cada transición. Para evaluar la cadena vacía, presionar ENTER sin escribir epsilon ni ϵ . No se eliminan espacios de la entrada: un espacio ajeno al alfabeto provoca rechazo.

Con afnd_enunciado.txt, la entrada ab produce el siguiente recorrido y termina en un estado final:

```
[S0]                --(a)-->                [S1]
[S1]                --(b)-->                [S2]
Veredicto final: ACEPTADA
```

La entrada b termina en S0 y se rechaza. La entrada abx se rechaza antes de iniciar la traza porque contiene x. La cadena vacía se rechaza en este ejemplo; con afnd_inicial_final.txt se acepta.

```

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 10
Cadena (ENTER: cadena vacía): ab

--- TRAZA DE EJECUCIÓN: 'ab' ---
[S0] --(a)--> [S1]
[S1] --(b)--> [S2]
Veredicto final: ACEPTADA
Motivo: El estado final de la computación 'S2' pertenece a F.

```

```

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 10
Cadena (ENTER: cadena vacía): abx

--- TRAZA DE EJECUCIÓN: 'abx' ---
(Sin pasos: la cadena es vacía o fue rechazada antes de simular)
Veredicto final: RECHAZADA
Motivo: La cadena contiene símbolos fuera del alfabeto Sigma: ['x'].

```

8.4.1 Procesamiento por lote

Seleccionar 11 e ingresar Ejemplos/cadenas_fase2.txt después de cargar el AFND del enunciado. El archivo contiene ab, b, una línea vacía, abx, aab y bab. El resultado es seis cadenas, tres aceptadas y tres rechazadas.

Cada línea del archivo representa una cadena. Una línea vacía representa la cadena vacía. Un archivo de cero bytes no contiene cadenas y se informa como vacío. Las líneas del lote no se interpretan como comentarios ni como definiciones de autómatas.

8.4.2 Historial de la sesión

La opción 12 consulta el historial del autómata activo: cadena, veredicto, traza y motivo, con una vista resumida. Repetir una evaluación agrega un registro. Cargar otro autómata o cambiar de activo conserva ambos historiales. Quitar uno elimina solo sus datos; cerrar el programa elimina todos.

```

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 11
Archivo de cadenas (cada línea vacía representa epsilon): C:\Users\veget\Downloads\Proyecto_Lenguajes_Fase_2\Proyecto_Lenguajes_Fase_2\Ejemplos\cadenas_fase2.
txt
=== REPORTE DE EVALUACIÓN POR LOTE ===
'ab' -> ACEPTADA (El estado final de la computación 'S2' pertenece a F.)
'b' -> RECHAZADA (El estado final de la computación 'S0' NO pertenece a F.)
'ε (cadena vacía)' -> RECHAZADA (El estado final de la computación 'S0' NO pertenece a F.)
'abx' -> RECHAZADA (La cadena contiene símbolos fuera del alfabeto Sigma: ['x'].)
'aab' -> ACEPTADA (El estado final de la computación 'S2' pertenece a F.)
'bab' -> ACEPTADA (El estado final de la computación 'S2' pertenece a F.)

Total de cadenas evaluadas: 6
Aceptadas: 3
Rechazadas: 3

```

8.5 Trampa análisis y cambio de sesión

8.5.1 Completar un AFD

Cargar Ejemplos/afd_incompleto.txt con 2. La evaluación queda bloqueada mientras falten transiciones. Elegir 6 y responder s para agregar q_trampa. Si se responde n, el AFD permanece incompleto. Después de completarlo, a se acepta y b se rechaza.

En un AFND la trampa se genera automáticamente cuando la conversión alcanza $\emptyset$. Para comprobarlo, cargar Ejemplos/afnd_con_trampa.txt con 4 y consultar 8 y 9. S2 representa $\emptyset$ y ambos símbolos vuelven a S2.

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===

1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir

Opción: 2

Ruta del archivo AFD: C:\Users\veget\Downloads\Proyecto_Lenguajes_Fase_2\Proyecto_Lenguajes_Fase_2\Ejemplos\afd_incompleto.txt

Cargado: AFD_incompleto

AFD incompleto

```
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 4
Ruta del archivo AFND: C:\Users\veget\Downloads\Proyecto_Lenguajes_Fase_2\Proyecto_Lenguajes_Fase_2\Ejemplos\afnd_con_trampa.txt

Cargado: AFND_con_trampa
AFND válido (sin epsilon)
ESTADO | a | b
->q0 | {q0, q1} | {}
q1 | {} | {q2}
*q2 | {} | {q2}
-> inicial; * final; {} sin destinos
Conversión terminada.
MACROESTADO | ESTADOS DEL AFND
S0 | {q0}
S1 | {q0, q1}
S2 | {}
S3 | {q2}
ESTADO | a | b
-----
->S0 | S1 | S2
S1 | S1 | S3
S2 | S2 | S2
*S3 | S2 | S3

Leyenda: '->' estado inicial, '*' estado final, '-' transición no definida
```

8.5.2 Análisis estructural

La opción 13 muestra alcanzables, inaccesibles, finales alcanzables y vacuidad del lenguaje. Para probar un final inaccesible, crear un AFD con $Q = \{q_0, q_1, q_2\}$, alfabeto $\{a\}$, inicial q_0 y final q_2 . Definir q_0, a, q_1 ; q_1, a, q_1 ; q_2, a, q_2 . El análisis debe mostrar q_2 inaccesible y lenguaje vacío.

8.5.3 Administración de dos autómatas y salida

La opción 14 abre este submenú: 1 AFD manual, 2 archivo AFD, 3 AFND manual, 4 archivo AFND, 5 seleccionar activo, 6 quitar, 7 consultar espacios y 8 regresar. Las cargas ocupan un espacio libre y lo activan. Si ambos están ocupados, primero se debe quitar uno.

Para seleccionar o quitar se ingresa 1 o 2; ENTER cancela. Quitar requiere s y elimina solo ese espacio. Si era el activo, se selecciona el otro. La opción 15 cierra el programa. El procedimiento y las capturas nuevas se encuentran en 10.2 a 10.5.

8.5.4 Mensajes frecuentes

Una ruta inexistente impide leer el archivo. Un archivo que declara AFND debe cargarse con 4. Las entradas sin autómata activo piden cargar uno. Las opciones 7 y 9 requieren un AFND original; un AFD cargado directamente no tiene tabla de equivalencias de conversión.

```
=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 4
Ruta del archivo AFND: C:\Users\veget\Downloads\Proyecto_Lenguajes_Fase_2\Proyecto_Lenguajes_Fase_2\Ejemplos\error_sintaxis.txt
[!] Línea 8: sintaxis inválida: 'q0;a;q1'.
```

```
=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Cargar o crear otro autómata
15. Salir
Opción: 4
Ruta del archivo AFND: C:\Users\veget\Downloads\Proyecto_Lenguajes_Fase_2\Proyecto_Lenguajes_Fase_2\Ejemplos\error_epsilon.txt
[!] Línea 8: Transiciones epsilon fuera del alcance de esta fase.
```

9. Manual técnico

9.1 Organización de archivos y dependencias

Archivo	Clases o funciones principales	Responsabilidad
main.py	AplicacionAFD, EspacioAutomata, main	Coordinar menú y dos espacios independientes.
cargador.py	CargadorAutomata	Obtener datos desde archivo o teclado.
analizador.py	AnalizadorSintactico	Interpretar el formato y construir el modelo.
validador.py	ValidadorAutomata	Aplicar reglas de integridad y clasificación.
afd.py	AFD	Guardar el modelo determinista y analizar su estructura.

afnd.py	AFND	Guardar conjuntos de destinos y sus representaciones.
conversor.py	ConversorAFND	Construir el AFD y conservar equivalencias.
simulador.py	Simulador y ResultadoEvaluacion	Evaluar cadenas y administrar el historial.

El cargador utiliza al analizador; el analizador utiliza los modelos y el validador. El conversor recibe AFND y construye AFD. El simulador recibe AFD y genera ResultadoEvaluacion. AplicacionAFD mantiene esos objetos y decide cuándo utilizarlos. El código no depende de un autómata específico.

9.1.1 Expresiones regulares vigentes

```

CABECERA          = (NOMBRE|TIPO|ESTADOS|ALFABETO|INICIAL|FINALES)\s*[:=]\s*(.*)
BLOQUE            = TRANSICIONES\s*:\s*
TRANSICION        = ([^,\s]+\s*,\s*([^\s,]*)\s*,\s*([^\s,]*)\s*,\s*(.??)
IDENTIFICADOR = [A-Za-z_][A-Za-z0-9_]*

```

CABECERA y BLOQUE se compilan con re.I. fullmatch exige que coincida toda la línea. El contenido de destinos se revisa después: debe ser $\emptyset$ o una lista de identificadores separada por |. La longitud y pertenencia del símbolo se validan con reglas adicionales. La expresión regular por sí sola no garantiza un autómata válido.

Las etiquetas no distinguen mayúsculas y minúsculas. Los nombres de estados y los símbolos sí conservan su escritura: q0 y Q0 son identificadores diferentes.

9.2 Control de la aplicación

AplicacionAFD conserva espacios = [None, None] y activo = None. Cada EspacioAutomata guarda original, afd_actual, conversor y simulador. En un AFD, ambos modelos son el mismo objeto; en un AFND, afd_actual es el equivalente generado.

Método	Función
__init__	Crea dos espacios vacíos, sin activo, y el cargador y validador.
cargar	Selecciona ingreso manual o archivo; instala únicamente una carga exitosa.
instalar	Ocupa un espacio libre, lo activa y conserva intacto el otro.
espacio_activo	Devuelve el espacio activo o lanza ValueError si no existe.
exigir_afd	Comprueba que haya un AFD válido y completo para la operación.
mostrar	Imprime definición y tabla del original y del equivalente cuando corresponde.
validar	Presenta el reporte y ofrece completar un AFD incompleto.
convertir	Comprueba que el original sea AFND y crea el equivalente solo si falta.
equivalencias	Imprime la tabla del conversor.
tabla_generada	Muestra la tabla del AFD obtenido de un AFND.
evaluar y lote	Solicitan entrada y delegan al simulador.
historial y analisis	Muestran el historial o el reporte estructural.
otro	Abre el submenú de ocho opciones para administrar los espacios.
ejecutar	Mantiene el bucle, busca acciones y controla errores previstos.

El diccionario acciones almacena referencias a métodos. Las expresiones lambda permiten llamar a cargar con un tipo y un origen determinados. La opción 15 se atiende antes de buscar una acción. ValueError, OSError y UnicodeError se muestran como mensajes; EOFError y KeyboardInterrupt finalizan el bucle principal.

La función main crea la aplicación y llama a ejecutar. La condición `__name__ == "__main__"` inicia el menú al ejecutar el archivo, pero no al importarlo desde otra parte del proyecto.

9.2.1 Espacios y selección del autómeta

EspacioAutomata.__init__ guarda los cuatro objetos. crear construye un conversor y un simulador propios, y convierte si recibe un AFND. La propiedad tipo devuelve AFD o AFND. mostrar_espacios enumera los dos lugares e indica el activo. _espacio_libre busca la primera posición vacía. seleccionar valida que el espacio exista y esté ocupado antes de cambiar activo.

quitar pide confirmación, asigna None al lugar elegido y, si era el activo, selecciona el restante o deja activo en None. El archivo de entrada no se elimina.

9.3 Carga y análisis sintáctico

9.3.1 CargadorAutomata

Método	Función
__init__	Crea un AnalizadorSintactico.
desde_archivo	Lee con utf-8-sig, que admite BOM, y devuelve (automata, errores).
_pedir_conjunto	Solicita una lista, elimina duplicados y repite la pregunta cuando la entrada es inválida.
manual	Valida inicial, finales y destinos, y construye AFD o AFND según el tipo.

Para validar el alfabeto manual se utiliza un AFD temporal mínimo y se reutilizan las reglas del validador. El modelo temporal no sustituye la sesión. La cancelación durante la creación devuelve None.

Las funciones leer_afd_desde_archivo y leer_afd_manual conservan accesos de la fase 1 y delegan al nuevo cargador. El menú actual utiliza la clase directamente.

9.3.2 AnalizadorSintactico

lista divide por comas, elimina espacios alrededor de cada elemento, verifica identificadores cuando corresponde y devuelve un set. Una entrada completamente vacía representa el conjunto vacío. Un elemento vacío intermedio, como a,,b, se considera error.

analizar procesa líneas, verifica cabeceras obligatorias y tipo esperado, forma conjuntos y revisa transiciones. Las líneas repetidas para un mismo par se acumulan con setdefault y update; no se pierde un destino anterior.

Si el tipo es AFND, todos los valores de transiciones son conjuntos. Si es AFD, se almacena un destino por par y se registran los pares con varios destinos en transiciones_conflictivas. Ese registro bloquea la evaluación: no se simula eligiendo arbitrariamente uno de los destinos.

9.3.3 Errores y retorno

El análisis devuelve (None, errores) cuando encuentra un problema que impide instalar la definición. De lo contrario devuelve (automata, []). Se comprueban primero estructura y sintaxis, después conjuntos y referencias. Esto explica por qué un archivo con problemas en varias etapas puede mostrar primero solo los errores sintácticos.

9.4 Modelos AFD y AFND

9.4.1 Clase AFD

Método	Función
<code>__init__</code>	Copia los componentes de la quintupla.
<code>validar_quintupla</code>	Delega integridad a <code>ValidadorAutomata.errores</code> .
<code>verificar_determinismo</code>	Revisa completitud y conflictos; devuelve booleano y mensajes.
<code>completar_con_estado_trampa</code>	Valida, elige un nombre libre, completa faltantes y agrega bucles.
<code>estados_alcanzables</code>	Recorre las transiciones desde el inicial con una cola e índice.
<code>estados_inaccesibles</code>	Devuelve Q menos los alcanzables.
<code>estados_finales_alcanzables</code>	Devuelve finales intersectados con alcanzables.
<code>lenguaje_vacio</code>	Comprueba que no exista un final alcanzable.
<code>quintupla_formal</code> y <code>tabla_transicion</code>	Construyen representaciones textuales del modelo.
<code>generar_reporte_validacion</code>	Reúne clasificación, integridad, completitud y análisis.
<code>__str__</code>	Permite imprimir la quintupla mediante <code>print(afd)</code> .

`verificar_determinismo` conserva el nombre de la fase 1, pero también revisa transiciones faltantes. Por eso un resultado falso debe interpretarse junto a los mensajes. El reporte reutiliza un único cálculo de alcanzabilidad para obtener sus conjuntos derivados.

9.4.2 Clase AFND

`__init__` copia los conjuntos y cada colección de destinos. `destinos` devuelve un conjunto vacío si el par no aparece. `mostrar_conjunto` transforma los estados en una representación ordenada o en $\emptyset$. `quintupla_formal` y `tabla_transicion` muestran el modelo original; no realizan la conversión.

`mostrar_conjunto` es estático porque no depende de los atributos de una instancia. `ConversorAFND` también lo utiliza al mostrar equivalencias. No se usa herencia entre los dos modelos.

ErrorValidacionAFD quedó definido en afd.py por continuidad con la primera fase. No participa en el flujo actual, que utiliza ValueError para bloquear operaciones inválidas.

9.5 Conversión y validación

9.5.1 ConversorAFND

`__init__` crea equivalencias vacío. `convertir` verifica el tipo y la integridad del AFND, inicializa `S0` con `{q0}` y recorre pendientes con un índice. Para cada símbolo construye la unión de destinos, registra subconjuntos nuevos y guarda una transición determinista.

El diccionario `nombres` utiliza `frozenset` como clave y el nombre `S` como valor. `equivalencias` invierte esa relación para consulta. Un conjunto se marca final si su intersección con `estados_finales` no está vacía. El resultado es una instancia de AFD con inicial `S0` y el mismo alfabeto.

`tabla_equivalencias` recorre los macroestados en el orden de descubrimiento y muestra sus conjuntos. La conversión no realiza minimización: dos macroestados diferentes se mantienen aunque pudieran ser equivalentes en un AFD mínimo.

9.5.2 ValidadorAutomata

Método	Función
<code>errores</code>	Revisa <code>Q</code> , inicial, finales, símbolos y referencias; no confunde incompletitud con invalidez.
<code>clasificar</code>	Distingue definición inválida, AFND válido, definición no determinista, AFD incompleto y AFD completo.
<code>exigir_afd</code>	Lanza <code>ValueError</code> si hay errores, si recibe AFND o si existen conflictos. Por defecto exige completitud.

La llamada `exigir_afd` con `completo=False` se utiliza para completar un AFD parcial. Permite transiciones faltantes, pero sigue rechazando referencias inválidas o no determinismo. Los marcadores epsilon detectados son la cadena vacía, epsilon, `eps`, ϵ , `landa`, λ y `lambda`.

9.5.3 Separación de responsabilidades

El analizador verifica que el texto pueda interpretarse y que las referencias declaradas tengan sentido. El validador comprueba el modelo también cuando se construye por otra vía. Esa segunda comprobación protege el conversor y el simulador frente a objetos inválidos creados directamente desde código.

9.6 Simulación y resultados

9.6.1 Clase Simulador

Método	Función
<code>__init__</code>	Inicializa historial como lista vacía.
<code>evaluar_cadena</code>	Exige AFD válido y completo, verifica símbolos, recorre delta y registra el resultado.
<code>_imprimir_traza</code>	Muestra movimientos, veredicto y motivo.
<code>evaluar_lote</code>	Lee cada línea y llama a <code>evaluar_cadena</code> con <code>mostrar_traza=False</code> .
<code>generar_reporte_lote</code>	Construye el detalle y cuenta aceptadas y rechazadas.
<code>mostrar_historial</code>	Enumera los resultados de la sesión o informa que está vacío.

`evaluar_cadena` devuelve un `ResultadoEvaluacion` y también lo agrega al historial. Una cadena inválida por alfabeto produce una traza vacía. El código conserva una comprobación defensiva de transición inexistente, aunque el flujo normal exige que el AFD esté completo antes de simular.

`evaluar_lote` elimina solamente los saltos de línea. No borra espacios ni omite líneas vacías. Un archivo inexistente, ilegible o sin contenido produce advertencias. La presentación del lote muestra resultados resumidos; las trazas permanecen dentro de cada resultado.

9.6.2 Clase ResultadoEvaluacion

Atributo	Contenido
<code>cadena</code>	La entrada evaluada, incluso si está vacía.
<code>aceptada</code>	Booleano con el resultado de aceptación.
<code>traza</code>	Lista de movimientos estado, símbolo y destino.
<code>motivo</code>	Explicación del resultado o del símbolo inválido.

`__init__` guarda los cuatro atributos. `__str__` genera el resumen que se utiliza en el lote y en el historial. El historial es una lista de objetos `ResultadoEvaluacion` y existe solo en memoria; no se escribe automáticamente en un archivo.

9.7 Eficiencia límites y mantenimiento

Sea n la cantidad de estados originales, k la cantidad de símbolos y m la longitud de una cadena. Las consultas de conjuntos y diccionarios tienen costo promedio constante para pertenencia o acceso por clave, sin contar el tamaño de las claves compuestas.

Operación	Costo y observación
Alcanzabilidad	$O(n \times k)$ en un AFD completo; cada estado alcanzable se procesa una vez.
Conversión	Se unen destinos para cada par macroestado y símbolo. En el peor caso pueden aparecer hasta 2^n macroestados.
Construcción de un destino	Puede recorrer hasta n estados con hasta n destinos cada uno; límite simple $O(n^2)$ por par en un AFND denso.
Simulación después de validar	$O(m)$ para revisar y recorrer la cadena; la traza utiliza memoria proporcional a m .
Validación previa	También recorre estados, símbolos y transiciones. El costo total de evaluar no es solo $O(m)$.
Lotes e historial	Cada espacio conserva sus resultados y trazas hasta quitar ese autómata o cerrar el programa.

9.7.1 Límites del formato

La implementación lee un carácter por símbolo. No admite símbolos de varias letras, espacios ni separadores reservados. No implementa transiciones epsilon ni minimización. Se conservan hasta dos autómatas en memoria, sin persistencia al cerrar. Estos límites se mantienen entre carga, validación y simulación.

9.7.2 Verificación después de cambios

Después de modificar el código, repetir la carga del AFD anterior, la conversión del AFND del enunciado, el caso con $\emptyset$ y las entradas inválidas. Comprobar además la cadena vacía y el cambio de sesión. Si se conserva la carpeta Pruebas, ejecutar desde la raíz:

```
python Pruebas/pruebas.py
```

Ese archivo es auxiliar y no se necesita para abrir el menú. Su comparación explora caminos del AFND de manera independiente del conversor. Si se omite del repositorio, la tabla de comparación de la sección 7.2 conserva una demostración reproducible de resultados sobre ambos modelos.

Para verificar los dos espacios, ejecutar los casos de la sección 10.2. Comprobar que seleccionar no modifica los modelos ni sus historiales y que completar un AFD solo cambia el espacio activo. Si se utilizan pruebas automatizadas anteriores, adaptar las que esperaban reemplazar la sesión.

10. Capturas y referencias

10.1 Evidencia visual por incorporar

Sustituir cada indicación CAPTURA PENDIENTE por una imagen legible de la ejecución. Las imágenes deben mostrar el archivo o la entrada utilizada y el resultado correspondiente. Evitar capturar rutas personales o áreas de escritorio ajenas al programa.

Captura	Ubicación	Contenido
1	Sección 8 1	Menú completo con las quince opciones.
2	Sección 8 2	AFND del enunciado, equivalencias y AFD generado.
3	Sección 8 4	Traza de ab y rechazo de abx.
4	Sección 8 4	Lote con 6 evaluadas, 3 aceptadas y 3 rechazadas; historial.
5	Sección 8 5	Completar AFD con trampa o conversión que produce $\emptyset$.
6	Sección 7 2	Aceptación y rechazo usados en la comparación AFND y AFD.
7	Sección 8 5	Mensaje de referencias inexistentes o epsilon fuera del alcance.

Para insertar una imagen, seleccionar la indicación correspondiente, eliminarla e ir a Insertar > Imágenes. Usar la disposición En línea con el texto, conservar la proporción y ajustar el ancho al área útil de la página. Después actualizar el índice completo. Los diagramas ya están incluidos y no necesitan capturas adicionales.

10.1.1 Capturas de la administración de autómatas

Se conservan las capturas ya incorporadas. Las capturas 8, 9 y 10 tienen espacios reservados en las secciones 10.3, 10.4 y 10.5, páginas 36, 37 y 38. Muestran el menú actualizado, el cambio de activo con historial y la eliminación seguida de una nueva carga. El menú visible en las capturas anteriores corresponde a la versión previa a la administración de dos espacios.

10.2 Casos de prueba de los dos espacios

Estos casos complementan CP10 y comprueban la administración descrita en 8.5.3. La tabla indica resultados esperados; las capturas siguientes permiten registrar la ejecución.

Caso	Acción	Resultado esperado
CP13	Cargar un AFND y un AFD; repetir con dos AFD y con dos AFND	Dos espacios ocupados; la última carga queda activa
CP14	Evaluar en cada uno y cambiar de activo con 14 > 5	Cada uno conserva su historial y, si es AFND, sus equivalencias
CP15	Intentar cargar un tercero	Se bloquea la carga sin reemplazar ninguno
CP16	Quitar un espacio y responder n; repetir con s	n conserva los datos; s elimina solo el espacio elegido
CP17	Quitar el activo con otro cargado; después quitar el restante	Se activa el otro; al quedar vacíos no hay activo
CP18	Con un espacio libre, cargar error_referencias.txt	Se informa el error; el otro espacio y el activo no cambian
CP19	Cargar otro después de quitar uno	Usa el primer espacio libre, queda activo y tiene historial vacío
CP20	Seleccionar un espacio vacío, ingresar 3 o presionar ENTER	Rechaza el espacio inválido; ENTER cancela la selección o eliminación
CP21	Completar afd_incompleto.txt con opción 6 y s	Solo modifica el AFD del espacio activo
CP22	Dejar ambos espacios vacíos y usar una consulta o evaluación	Solicita cargar un autómata y conserva el control del menú

10.3 Carga simultánea y menú actualizado

Captura 8 Dos autómatas cargados y opción 14

1. Iniciar el programa con ambos espacios vacíos
2. Elegir 4 y cargar afnd_enunciado.txt
3. Elegir 2 y cargar ejemplo_afd.txt
4. Entrar a 14 y mostrar el submenú completo junto con los dos espacios

Resultado esperado: espacio 1 con el AFND, espacio 2 con el AFD y este último marcado ACTIVO. El submenú muestra 5 seleccionar, 6 quitar, 7 consultar y 8 regresar.

En la carpeta actual del proyecto los .txt están junto a Programa. Desde Programa, usar ../afnd_enunciado.txt y ../ejemplo_afd.txt. Desde la raíz, usar solo el nombre del archivo. Si se usa otra ubicación, ingresar su ruta completa.

```
AFD valido y completo

AUTÓMATAS CARGADOS
Espacio 1: AFND_EJEMPLO [AFND]
Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD] (ACTIVO)

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Administrar autómatas
15. Salir
Opción: 14

AUTÓMATAS CARGADOS
Espacio 1: AFND_EJEMPLO [AFND]
Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD] (ACTIVO)

1. Cargar AFD manualmente
2. Cargar AFD desde archivo
3. Cargar AFND manualmente
4. Cargar AFND desde archivo
5. Seleccionar autómata activo
6. Quitar autómata
7. Consultar espacios
8. Regresar al menú principal
Opción: 
```

10.4 Selección e historiales independientes

Captura 9 Cambiar de activo y recuperar su historial

1. Con el AFND del enunciado activo, evaluar ab con la opción 10: ACEPTADA
2. En 14 > 5, seleccionar el espacio del AFD y usar 8 para regresar
3. Evaluar a con 10 y consultar su historial con 12
4. Volver al AFND con 14 > 5, regresar con 8 y consultar 12

Resultado esperado: el AFND conserva ab; la evaluación de a pertenecer solamente al AFD. Las opciones 8 y 9 del menú principal recuperan las equivalencias y la tabla del AFND activo. Los números 5 y 8 del procedimiento anterior son del submenú 14. Motivo: El estado final de la computación 'q1' NO pertenece a F.

```
AUTÓMATAS CARGADOS
Espacio 1: AFND_EJEMPLO [AFND]
Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD] (ACTIVO)

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Administrar autómatas
15. Salir
Opción: 12
=== HISTORIAL DE EVALUACIONES DE LA SESIÓN ===
1. 'a' -> RECHAZADA (El estado final de la computación 'q1' NO pertenece a F.)

AUTÓMATAS CARGADOS
Espacio 1: AFND_EJEMPLO [AFND]
Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD] (ACTIVO)
```

AUTÓMATAS CARGADOS

Espacio 1: AFND_EJEMPLO [AFND] (ACTIVO)

Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD]

=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===

1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Administrar autómatas
15. Salir

Opción: 12

=== HISTORIAL DE EVALUACIONES DE LA SESIÓN ===

1. 'ab' -> ACEPTADA (El estado final de la computación 'S2' pertenece a F.)

AUTÓMATAS CARGADOS

Espacio 1: AFND_EJEMPLO [AFND] (ACTIVO)

Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD]

10.5 Capacidad eliminación y nueva carga

Captura 10 Liberar y reutilizar un espacio

1. Con ambos espacios ocupados, intentar otra carga con 2 o 4
2. Entrar a 14 > 6, elegir el espacio que se quitará y responder s
3. Cargar afnd_inicial_final.txt con 4 dentro del submenú
4. Consultar los espacios con 7 y regresar al menú principal con 8

Resultado esperado: el tercer autómata se bloquea hasta liberar un lugar. La nueva carga ocupa ese lugar y queda activa. El autómata conservado mantiene su historial y equivalencias. Quitar un autómata no elimina el archivo .txt.

```
=== MOTOR DE AUTÓMATAS: FASES 1 Y 2 ===
1. Crear AFD manualmente
2. Cargar AFD desde archivo
3. Crear AFND manualmente
4. Cargar AFND desde archivo
5. Definición formal y tabla del autómata
6. Validar estructura / completar AFD con trampa
7. Convertir AFND a AFD
8. Tabla de equivalencias de macroestados
9. Tabla del AFD generado
10. Evaluar cadena
11. Evaluar archivo de cadenas
12. Historial de evaluaciones
13. Análisis estructural del AFD
14. Administrar autómatas
15. Salir
Opción: 2

[!] Los dos espacios están ocupados. Quite uno antes de cargar otro.

AUTÓMATAS CARGADOS
Espacio 1: AFND_EJEMPLO [AFND] (ACTIVO)
Espacio 2: Multiplos_de_3_en_binario_simplificado [AFD]
```

```

1. Cargar AFD manualmente
2. Cargar AFD desde archivo
3. Cargar AFND manualmente
4. Cargar AFND desde archivo
5. Seleccionar autómata activo
6. Quitar autómata
7. Consultar espacios
8. Regresar al menú principal
Opción: 4
Ruta del archivo AFND: C:\Users\veget\OneDrive\Escritorio\Proyecto_Lenguajes_Sebastian_y_Alessandro\Proyecto_Lenguajes\afnd_inicial_final.txt

Cargado en el espacio 2: AFND_inicial_final
AFND válido (sin epsilon)
ESTADO | a      | b
->*q0   | {q0, q1} | ∅
q1     | {q1}     | ∅
-> inicial; * final; ∅ sin destinos
Conversión terminada.
MACROESTADO | ESTADOS DEL AFND
S0           | {q0}
S1           | {q0, q1}
S2           | ∅
ESTADO      | a      | b
-----
->*S0        | S1      | S2
*S1         | S1      | S2
S2          | S2      | S2

Leyenda: '->' estado inicial, '*' estado final, '-' transición no definida

AUTÓMATAS CARGADOS
Espacio 1: AFND_EJEMPLO [AFND]
Espacio 2: AFND_inicial_final [AFND] (ACTIVO)

```